



HASHDIT

HashDit

Smart Contract Security

ListaDao MoolahVaultAccount **Security Audit Report**

PancakeSwap

20 Aug 2026 - 25 Aug 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	19 Aug 2026 - 25 Aug 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/229
Commit	feat/moolah-vault-account

Smart Contract List

No.	Contract Name	Verdict	Details
1	MoolahVaultAccount.sol	Medium	See Findings
2	deploy_moolahVaultAccount.s.sol	Green	-
3	deploy_moolahVaultAccount_impl.s.sol	Green	-

Findings

[M01] Aggregate vault NAV lets unrealized market losses be harvested as yield

Contract MoolahVaultAccount.sol

Severity Medium

Description

MoolahVaultAccount holds shares of a single MoolahVault whose lisUSD position is spread across 7 separate Moolah markets, confirmed on chain by a withdraw queue length of 7 and an aggregate totalAssets of roughly 28,493,462 lisUSD. The account measures harvestable yield as claimableYield = max(0, totalAssets - principal), where totalAssets reads the vault's aggregate value across all 7 markets after interest accrual.

That aggregate figure could include interest that has accrued on borrower positions which are underwater but have not yet been liquidated (in the future or existing if present), and it nets gains in healthy markets against losses forming in others. In the Moolah market model, bad debt is subtracted from a market's supply assets only at the moment a liquidation or write off is executed. Before that point the aggregate NAV is overstated, so claimableYield is overstated by the same amount.

Description

JavaScript

```
uint256 badDebtShares;
uint256 badDebtAssets;
if (position[id][borrower].collateral == 0) {
    badDebtShares = position[id][borrower].borrowShares;
    badDebtAssets = UtilsLib.min(
        market[id].totalBorrowAssets,
        badDebtShares.toAssetsUp(market[id].totalBorrowAssets,
market[id].totalBorrowShares)
    );

    market[id].totalBorrowAssets -= badDebtAssets.toUint128();
    market[id].totalSupplyAssets -= badDebtAssets.toUint128();
    market[id].totalBorrowShares -= badDebtShares.toUint128();
    position[id][borrower].borrowShares = 0;
}
```

Impact

Because claimYield lets the BOT withdraw and distribute the full apparent surplus to whitelisted recipients in one call, with no cap, delay, or solvency margin, the account can pay out value that is really soon to be lost principal. When the bad debt is later realized, the aggregate NAV falls below principal, but the surplus has already left the contract irreversibly.

	The fixed principal baseline does not prevent this, because at claim time the NAV genuinely reads above principal, and the existing liquidity and delivery checks validate withdrawal mechanics, not solvency. It multiplies the independent sources of latent loss, and a gain in one market can mask a loss building in another until that loss is realized.
Recommendation	Consider adding a solvency buffer so that only surplus above principal + buffer is claimable, where buffer is a MANAGER settable cushion expressed as an absolute amount or as a fraction of principal. This defers harvesting until the aggregate NAV clears the baseline plus a margin, so a moderate amount of unrealized loss can no longer be distributed as yield, and any yield held back inside the band becomes claimable later once the NAV clears it. If not, positions needs to be monitored carefully to prevent incorrect yield claim during bad debt within lisUSD moolah markets incurred on the MoolahVaultAccount
Status	Acknowledged, ListaDao has indicated that the following two operational steps is performed to prevent the issue: • Before every claim it scans the vault's markets for underwater positions and skips the harvest until the loss is realized • It claims at most claimableYield with a reserve (several days of accrued yield), keeping a first-loss cushion inside the position, which is economically equivalent to the suggested buffer.

[I01] increasePrincipal does not enforce the pairing of share transfer and baseline increase

Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description MoolahVaultAccount.increasePrincipal/assets) raises the accounting baseline (the principal) but does not itself move the vault shares that the new baseline is meant to represent. The intended safe usage, as the contract header notes (4 to 5), is a single Safe MultiSend batch that transfers vault shares into the contract and calls increasePrincipal in the same transaction. Nothing on-chain enforces that these two actions happen together. <pre> JavaScript function increasePrincipal(uint256 assets) external onlyRole(MANAGER) { require(assets > 0, ZeroAmount()); principal += assets; emit PrincipalIncreased(assets, principal); </pre>

```
}
```

Impact

The impact is that any post launch share top up executed as two separate Safe transactions leaves a window of risk. Because a plain ERC20 share transfer has no hook, this is a realistic operator slip. During the interim between the share transfer and the increasePrincipal call, the entire transferred corpus is read as claimableYield. If a single claimYield from the BOT lands in that window, the whole top up is paid out as "yield" to a whitelisted address, and the loss is irreversible.

This is the largest fund at risk vector in the contract, and its only current mitigation is off chain process discipline.

Recommendation

Consider adopting an atomic entry point that both pulls the shares and updates the baseline in one call the contract itself guarantees, rather than relying on MultiSend batch discipline.

Status

Acknowledged, ListaDao has indicated that a share top-up and its increasePrincipal is always enforced in ONE Safe MultiSend.

[I02] MANAGER and principalOwner being the same multisig collapses the principal and yield separation under a MANAGER multisig compromise

Contract MoolahVaultAccount.sol

Severity **Informational**

Description

The MoolahVaultAccount design emphasizes two protections: MANAGER cannot redirect principal because setPrincipalOwner is admin only, and yield cannot reach the corpus. Both statements are accurate on their own. However, MANAGER, principalOwner, and the fund owner are all the same B0c6 multisig, so a compromise of that single multisig drains everything regardless.

Description

```
JavaScript
address admin = 0x07D274a68393E8b8a2CCf19A2ce4Ba3518735253; // protocol
TimeLock
address manager = 0x8d388136d578dCD791D081c6042284CED6d9B0c6; // B0c6
address bot = 0x91fC4BA20685339781888eCA3E9E1c12d40F0e13;
address pauser = 0xEEfebb1546d88EA0909435DF6f615084DD3c5Bd8;

address vault = 0xE03D86e5Baa3509AC4A059A41737bAa8169B6529; // lisUSD
MoolahVault
address principalOwner = 0x8d388136d578dCD791D081c6042284CED6d9B0c6;
// B0c6
```

	Impact An attacker who controls that multisig can extract principal through withdrawPrincipal or emergencyWithdraw, which send to B0c6 (now controlled by the attacker), and can also extract yield by self whitelisting and self granting the BOT role. The practical consequence is that the principal and yield separation only provides defense in depth against a compromised BOT or PAUSER, not against a compromised MANAGER key. This behavior is documented in note 3.
Recommendation	Consider assigning MANAGER, principalOwner, and the fund owner to distinct keys or multisig addresses so that a single key compromise cannot both move principal and self grant yield access
Status	Acknowledged, ListaDao has indicated that this is by design, wherein the principal/yield separation is defense-in-depth against a compromised BOT or PAUSER key, not the MANAGER.

[I03] Missing event emission for principalOwner and principal assignment in initialize	
Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The MoolahVaultAccount.initialize() function sets the principalOwner and principal but does not emit an event when it does so. <pre> JavaScript principalOwner = _principalOwner; principal = _principal; for (uint256 i = 0; i < _yieldRecipients.length; ++i) { _addYieldRecipient(_yieldRecipients[i]); } </pre> Impact Because this initial assignment is a significant configuration action that establishes who controls the principal and principal amount, the absence of an event makes it harder for off chain monitoring tools, indexers, and users to track when and to what value principalOwner was first set.
Recommendation	Consider emitting the dedicated SetPrincipalOwner and PrincipalDeposited event within the initialize function

Status	Acknowledged, the fix is optional given it does not affect core contract functionalities. Indexers can retrieve the initialize transaction to retrieve the initial principalOwner and principal initialized.
---------------	--

[I04] No recovery path for excess or accidentally donated underlying assets	
Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The MoolahVaultAccount contract does not provide a way to withdraw excess underlying assets or tokens that are accidentally sent to it. Impact If underlying assets are transferred directly to the contract outside of its normal accounting flow, or if the contract holds more of the underlying asset than its accounting expects, those funds have no dedicated recovery mechanism. As a result, such assets can become permanently stuck in the contract with no way to retrieve them.
Recommendation	Consider adding a permissioned recovery function that lets an authorized role withdraw excess or accidentally donated assets. The function should compute the recoverable amount as the difference between the contract's actual token balance and the amount tracked by internal accounting, so that only genuine excess is withdrawn and funds accounted for as principal or yield are not affected. Restrict this function to a trusted role and emit an event when a recovery occurs. If not, this can be deferred to a future upgrade if there are donated funds in the future.
Status	Acknowledged, ListaDao has indicated that a possible upgrade would be performed in the future if there are any material funds stuck in the MoolahVaultAccount in the future.

[I05] EmergencyWithdrawn event omits the reset principal value	
Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The MoolahVaultAccount contract keeps its event data consistent for principal movements where PrincipalDeposited, PrincipalWithdrawn, and PrincipalIncreased all emit the resulting principal value after the change. The emergencyWithdraw() function breaks this pattern. It sets principal to 0 as part of the emergency exit, but the EmergencyWithdrawn event it emits only carries the destination address and the share amount, leaving out the new principal value.

JavaScript

```
function emergencyWithdraw() external onlyRole(MANAGER) nonReentrant
returns (uint256 shares) {
    shares = vault.balanceOf(address(this));
    require(shares > 0, ZeroShares());

    principal = 0;

    IERC20(address(vault)).safeTransfer(principalOwner, shares);
    require(vault.balanceOf(address(this)) == 0, SharesRemaining());

    emit EmergencyWithdrawn(principalOwner, shares);
}
```

Impact

Because of this omission, an off chain indexer or monitoring system that reconstructs the current principal purely from emitted events cannot observe the transition of principal to 0 caused by an emergency withdrawal. This produces an inaccurate off chain view of the account state. The impact is limited to observability and event consistency, with no effect on on chain funds or accounting.

Recommendation

Consider extending the event to include the updated baseline, for example `EmergencyWithdrawn(address indexed to, uint256 shares, uint256 principal)`, and emit it with the post reset value of 0.

Status

Acknowledged. ListaDao has indicated that a event field addition will be deferred to a future upgrade, given `emergencyWithdraw` resets principal to 0 unconditionally, so the post-event value is a constant an indexer can infer from `EmergencyWithdrawn` itself.

[I06] Modifier order placement of `nonReentrant` before `whenNotPaused`

Contract MoolahVaultAccount.sol

Severity Informational

Description

Description

In `MoolahVaultAccount`, the functions `depositPrincipal()`, `withdrawPrincipal()`, and `claimYield()` declare their modifiers in the order `nonReentrant whenNotPaused`. Modifiers execute left to right, so `nonReentrant` runs first and engages the reentrancy guard (writing to its storage slot) before `whenNotPaused` is ever evaluated.

Impact

When the contract is paused, the call still enters and updates the reentrancy guard state, then reverts on the pause check, and the guard is rolled back on revert. This wastes gas on a call that was always going to fail and performs guard bookkeeping for an operation that should have been rejected up front on the cheaper, higher priority

	pause condition. The more natural ordering is to reject a paused contract before doing any reentrancy guard work.
Recommendation	Consider switching the modifier order to whenNotPaused nonReentrant
Status	Acknowledged, no fix implemented. This is optional considering a paused state is not common and modifier ordering does not affect core contract functionalities and the safeguards it provides.

[I07] Single step principalOwner migration in MoolahVaultAccount	
Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The MoolahVaultAccount.setPrincipalOwner() function overwrites principalOwner in a single step, with no confirmation from the incoming address. principalOwner is the only permitted destination for the two functions that move the protocol assets where withdrawPrincipal() sends the redeemed lisUSD to it, and emergencyWithdraw() transfers the entire vault share balance to it. Both rely on plain token transfers that succeed without any acknowledgment from the receiver. <pre> JavaScript function setPrincipalOwner(address _principalOwner) external onlyRole(DEFAULT_ADMIN_ROLE) { require(_principalOwner != address(0), ZeroAddress()); require(_principalOwner != principalOwner, AlreadySet()); principalOwner = _principalOwner; emit SetPrincipalOwner(_principalOwner); } </pre> Impact If the DEFAULT_ADMIN_ROLE holder sets principalOwner to a wrong or uncontrolled address, nothing on chain verifies that the new owner can operate the funds, and the next withdrawPrincipal() or emergencyWithdraw() call will route the corpus to that address with no way to reverse it. The 24h TimeLock delay on setPrincipalOwner() provides a review window but does not prove that the target address is controllable.
Recommendation	Consider replacing the single step setter with a two step ownership transfer. Have setPrincipalOwner() store the proposed address as a pending value, and require that address to call a separate acceptance function such as acceptPrincipalOwner() before it becomes the active principalOwner.

	Alternatively ensure that a principal owner switch is monitored carefully given there is sufficient time period to do so before adjustment.
Status	Acknowledged, no fix implemented. The setPrincipalOwner sits behind ListaDao's timelock contract with a minimum delay of 24 hours.

[I08] claimYield() lets BOT route the entire yield to a single whitelisted recipient with no MANAGER enforced split

Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The claimYield() function in MoolahVaultAccount allows the BOT role to freely decide how the harvested surplus is divided among the whitelisted destinations. The only constraints enforced on chain are that every destination is whitelisted and that the sum of the payments does not exceed claimableYield(). There is no on chain allocation ratio, so the BOT, which is a single externally owned account, can direct 100% of the accumulated yield to any one whitelisted address and 0% to the others. <pre> JavaScript function setPrincipalOwner(address _principalOwner) external onlyRole(DEFAULT_ADMIN_ROLE) { require(_principalOwner != address(0), ZeroAddress()); require(_principalOwner != principalOwner, AlreadySet()); principalOwner = _principalOwner; emit SetPrincipalOwner(_principalOwner); } </pre> Impact Currently, the two configured destinations serve distinct protocol purposes. The LSR pool (LisUSDPoolSet) receives lisUSD as raw backing that funds its duty based payout to stakers, and its staker withdrawals require the pool to physically hold enough lisUSD. The USDT buyback contract receives lisUSD to fund buybacks. Because the split is entirely a backend decision and nothing on chain requires a minimum share to each sink, a compromised or misconfigured BOT can starve one destination completely. Systematically routing all yield away from the LSR pool can leave it unable to back the rate it has promised stakers, while routing everything there starves the buyback program. The misallocation stays within the approved whitelist, so funds are not stolen, but they can be sent to the wrong protocol sink with no delay and no MANAGER approval on the proportion.
Recommendation	Introduce a MANAGER configurable allocation policy that claimYield() validates at execution time, for example a set of per recipient weights expressed in basis points

that must sum to 100%, with the payment amounts checked against those weights within a defined tolerance. This keeps the routing decision under **MANAGER** control while still letting the BOT trigger the harvest.

If the split is intentionally left as a BOT and backend decision, then confirm that the backend accurately claim the required yield amounts and document that each target contract supports non invasive migration and rebalancing, so that **lisUSD** delivered to one destination can be redirected to the correct sink without disruptive or privileged intervention. This ensures a misallocation by the BOT can be corrected after the fact rather than becoming stuck in a single destination.

Status

Acknowledged, ListaDao has indicated that the split between the LSR pool and the **USDT** buyback is a backend decision as backend allocation config is review-gated, so a misallocation is can be corrected by protocol ops rather than lost.

[I09] Yield recipient rotation requires add before remove ordering and can revert during an incident

Contract

MoolahVaultAccount.sol

Severity

Informational

Description

Description

The MoolahVaultAccount recipient whitelist enforces that at least one yield recipient always remains, so `removeYieldRecipient()` reverts with `NoYieldRecipient` when it would empty the list.

JavaScript

```
function increasePrincipal(uint256 assets) external onlyRole(MANAGER) {
    require(assets > 0, ZeroAmount());

    principal += assets;

    emit PrincipalIncreased(assets, principal);
}
```

Impact

As a result, the list can never be drained to zero and then refilled. To replace the full set of recipients, the **MANAGER** must first call `addYieldRecipient()` for the new destination or destinations and only then remove the old ones. If the **MANAGER** instinctively removes recipients first, the removal that would clear the final entry reverts. The situation is fully recoverable, since adding a new recipient before removing the last old one always succeeds, so no funds are at risk and no state is corrupted. The concern is operational, because this ordering requirement is a foot gun during an incident driven rotation, where an operator under time pressure may attempt to remove first and hit an unexpected revert.

Recommendation	Consider providing a single atomic replacement function, for example <code>replaceYieldRecipient(address oldRecipient, address newRecipient)</code>, that adds the new recipient and removes the old one in one call while preserving the at least one invariant throughout. If not, ensure that replacement of a complete list of whitelisted yield recipients is batched atomically called as mentioned above.
Status	Acknowledged, ListaDao has indicated that operationally, the add-first-then-remove step will be batched in a single Safe transaction.

[I10] depositPrincipal() trusts the vault to pull the exact deposit amount without a balance delta check

Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description The MoolahVaultAccount.depositPrincipal() function is the only fund moving path that does not re read balances to defend against a misbehaving upgradeable vault. Both <code>claimYield()</code> and <code>emergencyWithdraw()</code> verify the actual effect of their vault interactions by re reading balances afterward, but <code>depositPrincipal()</code> assumes <code>vault.deposit</code> pulls exactly assets and then executes <code>principal += assets</code> unconditionally after only checking that the returned share count is <code>> 0</code>. <pre> JavaScript /// @dev pull lisUSD from the caller and supply it to the vault, /// raising principal by the same /// nominal amount. Requires this contract on the vault's deposit /// whitelist. function depositPrincipal(uint256 assets) external onlyRole(MANAGER) nonReentrant whenNotPaused { require(assets > 0, ZeroAmount()); asset.safeTransferFrom(msg.sender, address(this), assets); asset.forceApprove(address(vault), assets); uint256 shares = vault.deposit(assets, address(this)); // a deposit too small to mint a share must not raise principal for free require(shares > 0, ZeroShares()); principal += assets; emit PrincipalDeposited(msg.sender, assets, principal); } </pre> Impact

	Because the vault is upgradeable, a future buggy or malicious vault implementation could pull less than assets while still returning shares > 0. In that case principal would over state the value actually supplied. The lisUSD that was not pulled would remain in the contract as stray balance with no rescue mechanism, since the contract intentionally has no sweep function. In addition, the forceApprove allowance granted to the vault would not be fully consumed, leaving a dangling approval to the vault. This requires a compromised or defective vault to trigger, so the practical risk is low, but it is inconsistent with the defensive balance checks applied on every other fund path.
Recommendation	Make depositPrincipal() verify the real effect of the deposit rather than trusting the vault. Measure the contract's lisUSD balance before and after the deposit and increase principal by the amount actually transferred out, or assert that the observed asset delta and minted share amount match the expected values, reverting otherwise. Additionally, consider reset the vault allowance to 0 after the deposit completes so no residual forceApprove allowance remains.
Status	Acknowledged, ListaDao has indicated that a balance-delta check + allowance reset indepositPrincipal will be included in the next implementation upgrade for consistency with the other fund paths.

[I11] Operational risk profile for a dominant MoolahVaultAccount position

Contract	MoolahVaultAccount.sol
Severity	Informational
Description	Description This finding lists operational realities that a holder of a large MoolahVaultAccount position must account for. None is a defect in the account contract itself, but at scale each can cause loss, stuck funds, or a mispriced position, and together they define how the position must be operated. Withdrawals and harvests cannot be assumed to work on demand. At scale the position can easily exceed the amount of loan token actually free in Moolah, because the rest is borrowed out. Both yield-facing exits walk the vault withdraw queue and are all-or-nothing per call. claimYield performs a single vault.withdraw(total), and if total exceeds current liquidity the call reverts with NotEnoughLiquidity rather than partially filling. withdrawPrincipal behaves the same way. A harvest or withdrawal must therefore be sized to previewClaim().claimableNow, never to claimable, because claimableNow = min(claimable, maxWithdraw) and maxWithdraw already nets out real Moolah liquidity. At high market utilization a large claimable can sit unharvestable for extended periods, so yield is not always available on demand. The one liquidity-independent full exit is emergencyWithdraw, which transfers shares to principalOwner, never touches Moolah, and works even when the queue is dry or Moolah is paused. For a position too large to redeem in one liquid step, the reliable pattern is emergencyWithdraw followed

by principalOwner redeeming shares on its own schedule. This should be treated as the primary full-exit route, not only an incident tool.

Supply Cap and Fund Transfer Flow

A large depositPrincipal needs enough aggregate supply cap across the vault's supplyQueue, or it reverts with AllCapsReached, so caps and whitelist status must be confirmed in advance and large deposits staged. Separately, when a corpus is recorded as vault shares, the flow is a plain ERC20 share transfer paired with increasePrincipal. At scale this pairing must execute as one atomic Safe MultiSend. If it is split across two transactions, the entire transferred corpus reads as claimableYield() in the gap, and a BOT harvest landing in that window pays the corpus out to a whitelisted address as yield. Nothing on-chain enforces the pairing, since a plain ERC20 share transfer has no hook to intercept, which makes this the single most dangerous moment, and its blast radius scales 1:1 with the position size.

Market Risk as Dominant Supplier

If the account is most of the vault and the vault is most of a market's supply, the account bears essentially all of that market's bad debt. Bad debt is socialized in Moolah as an immediate reduction of totalSupplyAssets by the bad debt amount, which is an instant NAV drop shared pro-rata across suppliers. Two consequences follow at this concentration. First, the risk taken is whatever markets the vault's CURATOR and ALLOCATOR point liquidity at, including the oracle, LLTV, and collateral of each market in the supplyQueue. The account does not choose these, the roles do, so the allocation must be vetted and monitored. Second, totalAssets() trusts convertToAssets, which counts interest accruing on borrows that may be uncollectable, meaning unrealized bad debt. NAV, and therefore claimableYield(), can be overstated before a liquidation realizes the loss. Harvesting in that window pays out phantom yield and can push the remaining position below principal, so harvests should be conservative before large claims rather than draining claimableNow to the wei.

Recommendation

- Size every harvest and withdrawal to claimableNow rather than claimable, and harvest conservatively so a NAV that includes unrealized bad debt does not cause a payout that drops the remaining position below principal.
- Treat emergencyWithdraw to principalOwner, followed by paced redemptions, as the primary full-exit route for any position too large to redeem in one liquid step, rather than as an incident-only tool.
- Before large deposits, confirm supply caps across the supplyQueue and whitelist status, and stage the deposits. Always execute the share transfer and increasePrincipal as a single atomic Safe MultiSend, never as two separate transactions, and consider an on-chain path that pulls shares and raises the baseline in the same call so the pairing cannot be split.

-
- Continuously vet and monitor the vault's market allocation, including each market's oracle, LLTV, and collateral, and monitor the governance levers, namely fee, feeRecipient, supplyQueue, withdrawQueue, caps, lockBuffer, and provider.

Status

Acknowledged, ListaDao has indicated that the recommendations has already been implemented and/or considered in the backend operational runbook.